

RTOS Fundamentals and FreeRTOS Orientation

A concept-first entry point before project structure, source files, and STM32 self labs.

Course baseline: NUCLEO-F439ZI, STM32CubeIDE, FreeRTOS kernel, CMSIS-RTOS V2 reading path



Module role

- Real-time vocabulary before API details
- FreeRTOS identity, history, and ecosystem
- Why an RTOS changes firmware structure
- Bridge into Module 01 project boundaries

Position of Module 00

The existing course already has a strong project and API path. This module supplies the conceptual base that makes the rest easier to read.

Concept layer

- RTOS meaning
- Real-time timing language
- Multitasking on a single core
- Kernel feature map

FreeRTOS layer

- Kernel and modular libraries
- Task terminology
- MIT license and ecosystem
- OpenRTOS and SafeRTOS family context

Course layer

- NUCLEO-F439ZI baseline
- STM32CubeIDE environment
- Self Lab entry point
- Module 01 project tree bridge

Embedded Firmware Before an RTOS

A small embedded program often begins as one control loop plus a few interrupt handlers.

```
int main(void)
{
    board_init();

    while (1) {
        read_sensors();
        update_control();
        service_uart();
        refresh_outputs();
        check_buttons();
    }
}
```

What works at first

- Low overhead
- Direct execution order
- Simple debug path
- No scheduler configuration

What starts to bend

- Slow calls delay unrelated work
- Timing dependencies become hidden
- Polling wastes CPU time
- ISR code can grow too much

When a Loop Stops Scaling

RTOS pressure appears when different parts of the firmware need different timing behavior.

Fast event

A button, sensor threshold, or packet arrival needs a short response path.

Periodic work

A control loop or sampling routine needs a stable release interval.

Slow work

Logging, formatting, flash access, or network work may run for longer than expected.

Background

Health checks and diagnostics should run when the CPU has slack.

Loop-level symptom

- The slowest path controls the response time of unrelated work.
- Interrupt handlers start carrying too much application logic.
- Adding one feature changes timing behavior elsewhere.
- Debug output changes timing enough to hide the original bug.

An RTOS gives the firmware a vocabulary for separating work by priority, state, and waiting condition.

Real-Time Means Predictable, Not Fast

A real-time system is evaluated by whether important work completes within meaningful timing constraints.

Latency

Delay between an event and the start or completion of the response that matters.

Jitter

Variation in timing from one execution to the next, even when the same code path runs.

Deadline

A time boundary after which the result becomes degraded, late, or useless.

Determinism

The ability to reason about timing behavior under a defined set of conditions.

A faster MCU can reduce average execution time. It does not automatically make response time predictable when control flow and priority rules are unclear.

Hard and Soft Real-Time Requirements

The Kernel Book introduces this distinction before any API because it shapes priority and scheduling decisions.

Hard real-time

- A missed deadline can mean system failure.
- Correctness includes timing, not only the computed value.
- Design tends to reserve priority and bounded paths for the critical work.
- Example category: protection, emergency response, safety action.

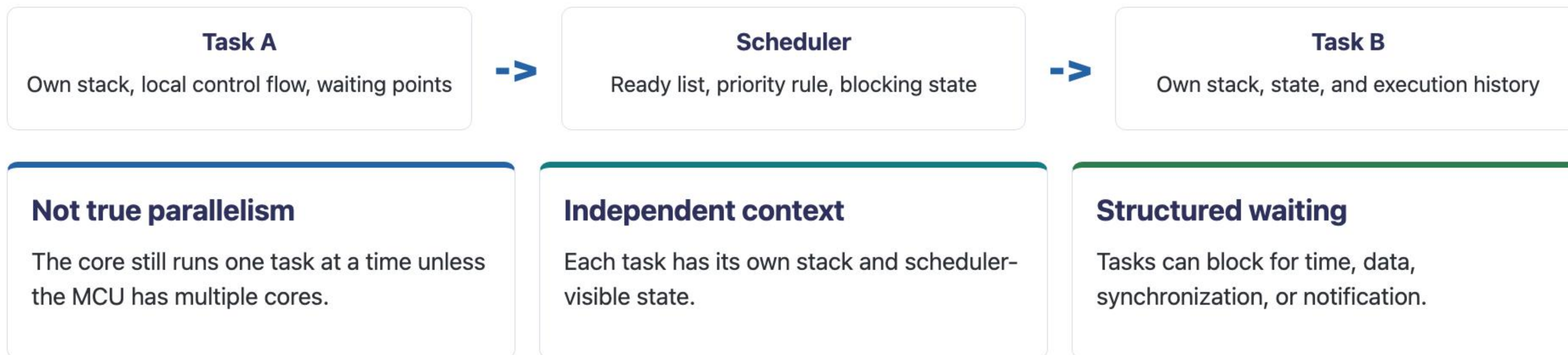
Soft real-time

- A missed deadline reduces quality or responsiveness.
- The system may remain usable after late completion.
- Design can tolerate occasional delay, buffering, or retry.
- Example category: user input, display update, status reporting.

Priority assignment is not a moral ranking of code. It is a timing decision made for a specific system behavior.

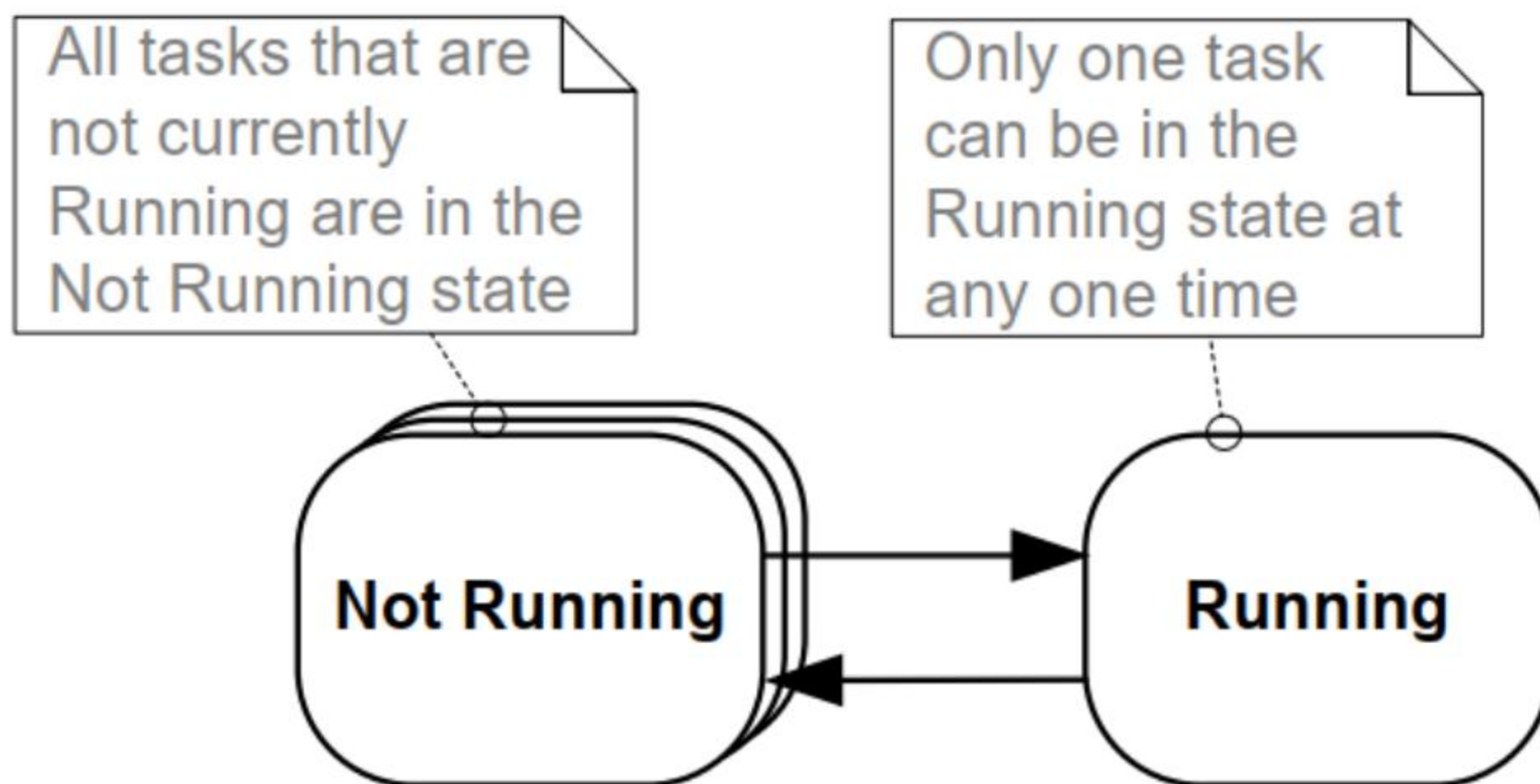
Multitasking on a Single Core

A Cortex-M4F core executes one instruction stream at a time. The RTOS makes multiple execution contexts manageable.



What the Scheduler Decides

The scheduler does not run every task. It selects among tasks that are eligible to run.



Preview vocabulary

- **Ready** : eligible for CPU selection
- **Running** : currently executing on the core
- **Blocked** : waiting for time, data, or an event
- **Suspended** : outside normal scheduling until resumed

Module 03 turns this preview into a full state transition model.

What FreeRTOS Is

FreeRTOS is not a desktop-style operating system. It is a small real-time kernel and a set of modular C libraries for microcontrollers and small microprocessors.

Application code Tasks, drivers, business logic, protocol decisions, product behavior.

FreeRTOS kernel Scheduler, task state, queues, semaphores, mutexes, timers, event groups, notifications.

Port and HAL/BSP Cortex-M port code, STM32 HAL, board support, interrupt vector, clock and peripheral setup.

Toolchain Compiler, startup file, linker script, generated firmware image, debug probe.



A compact kernel and library ecosystem for deeply embedded software.

Course framing

- FreeRTOS code is linked into the firmware image.
- The kernel runs because application code creates tasks and starts the scheduler.
- Board-specific details remain visible through the port, HAL, BSP, startup, and linker layers.
- STM32CubeIDE can generate a CMSIS-RTOS V2 wrapper path, but the native FreeRTOS concepts remain the foundation.

FreeRTOS in the Firmware Image

The first source-level idea: FreeRTOS becomes part of the firmware build, not a separate installed OS.

What appears in the project

- `main.c` and application task files
- `FreeRTOSConfig.h`
- Kernel sources such as `tasks.c` and `queue.c`
- Portable layer for ARM Cortex-M
- HAL/BSP and startup/linker files

Create objects

```
xTaskCreate()  
xQueueCreate()
```



Start scheduler

```
vTaskStartScheduler()  
or CMSIS wrapper path
```



Run by state

Ready,
Running,
Blocked,
interrupt
wakeup

Terminology: Task, Thread, Process

FreeRTOS calls each schedulable execution context a task. That choice avoids desktop OS assumptions.

Term	Typical meaning	Course usage
task	FreeRTOS execution unit with its own stack and scheduler-visible state.	The main term for Module 02 onward.
thread	General OS term for a schedulable execution path inside a process.	Used only for analogy when needed.
process	Usually implies an address space and OS-managed resource boundary.	Usually avoided for MCU FreeRTOS explanations.
kernel object	Queue, semaphore, mutex, timer, event group, notification, buffer.	Used to group FreeRTOS APIs by design purpose.

A task is not a separate program. On a typical Cortex-M FreeRTOS system, tasks share the same address space.

Why Use an RTOS?

The Kernel Book frames RTOS use as a set of software design benefits, not only a way to chase deadlines.

Timing abstraction

Delay, timeout, periodic release, and event wait become explicit API-level concepts.

Maintainability

Timing details move out of tangled module-to-module control flow.

Modularity

Tasks become independent modules with defined responsibility and interface.

Testing and reuse

Smaller independent execution paths are easier to test and move to another project.

Efficiency

Blocked tasks do not poll. They wake when time, data, or events arrive.

Idle time

The idle task exposes spare CPU capacity and low-power opportunities.

Interrupt discipline

ISRs can stay short by deferring processing to task context.

Event-Driven Firmware

A major RTOS shift is moving from repeated checking to blocked waiting and explicit wakeup.

Polling-heavy loop

- Repeated checks even when nothing changed
- CPU time spent asking the same question
- Long loop path increases response variation
- State scattered across flags and timing counters

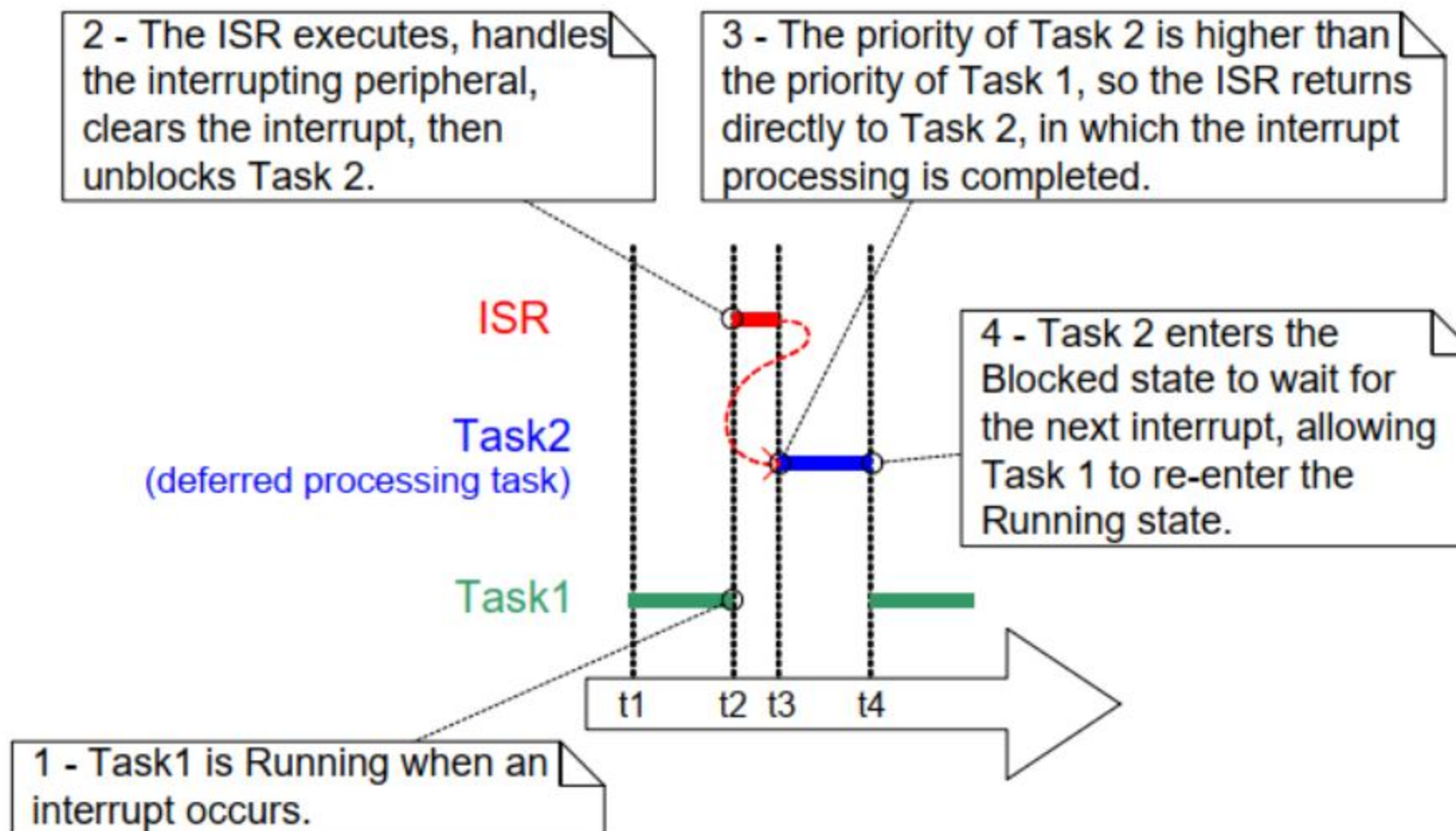
RTOS event-driven path

- Task blocks until data, time, or event is ready
- Scheduler runs another Ready task while waiting
- Queue, semaphore, notification, or event group encodes the wakeup reason
- Timeout policy becomes part of the design

Module 06 through Module 14 are mostly a study of different wakeup and communication objects.

Interrupts Stay Short

An RTOS does not remove interrupts. It gives the firmware a disciplined way to leave heavy work outside ISR context.



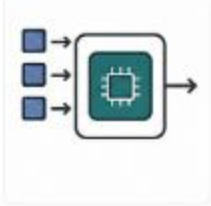
Preview rule

- ISR handles the immediate hardware event.
- ISR uses a `FromISR` API when FreeRTOS interaction is needed.
- A task completes longer processing in normal task context.
- The ISR may request a context switch if a higher priority task becomes Ready.

Module 08 will make this a concrete API and priority design problem.

FreeRTOS Kernel Feature Map

Module 00 only names the tools. Later modules explain mechanisms, API details, failure modes, and STM32 checkpoints.



Task

Independent execution context and responsibility boundary.



Scheduler

Priority and state based CPU ownership decision.



Queue

Data transfer plus blocking behavior between tasks.



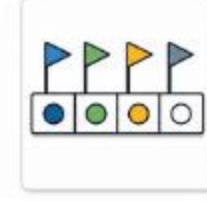
Mutex

Shared resource ownership with priority inheritance.



Timer

Time event callback without application polling loops.



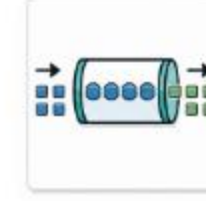
Event group

Multiple conditions encoded as waitable bits.



Notification

Lightweight direct signal to one task.



Buffers

Byte stream and message boundary data paths.

From Feature Names to Course Modules

The full course is organized around the problems each kernel object solves.

Problem	FreeRTOS concept	Detailed module
Separate execution responsibilities	Task, TCB, stack	Module 02
Choose execution order under timing pressure	Priority, tick, scheduler policy	Module 03-04
Move data between tasks	Queue, mailbox, pointer queue	Module 06-07
Signal events and protect resources	Semaphore, mutex, critical section	Module 10-11
Represent time-driven work	Software timer, daemon task	Module 09
Handle many conditions or lightweight signals	Event group, task notification	Module 12-13
Handle serial data and packets	Stream buffer, message buffer	Module 14
Know why the firmware fails	Stack, heap, runtime stats, trace	Module 05 and Module 15

FreeRTOS Value Proposition

FreeRTOS is widely used because the technical model is small, portable, commercially usable, and supported by a broad ecosystem.



Richard Barry

Creator of FreeRTOS. The course treats FreeRTOS as both a small kernel and a long-lived embedded ecosystem.

Photo source: Arm Newsroom

Around 2003

Richard Barry originally develops FreeRTOS.

Real Time Engineers

Development continues with close chip-company partnership.

AWS stewardship

AWS takes stewardship and maintains the project as open source.

MIT license

Use in personal and commercial projects without exposing proprietary source.

Current ecosystem

40+ processor architectures, libraries, LTS releases, community, partners.

Small

Designed for memory, compute, and power constrained embedded targets.

Portable

C source, architecture ports, and many toolchain integrations.

Neutral

Hardware, tool, and cloud-service neutral open-source project.

FreeRTOS, OpenRTOS, and SafeRTOS

The course uses open-source FreeRTOS, but the family context matters for industry discussions.

FreeRTOS

- MIT open-source license
- Commercial application use allowed
- Users retain their own intellectual property
- Main target for this course

OpenRTOS

- Commercially licensed version
- Provided through a third party under license
- Useful context for legal and procurement paths
- Not the lab target here

SafeRTOS

- Related usage model
- Developed for safety standard compliance claims
- Relevant to safety-critical product domains
- Not a shortcut to safety certification

Why NUCLEO-F439ZI for This Course

The board is not the subject of the course, but it gives the RTOS concepts a repeatable execution target.

Core lab value

- Cortex-M4F at the right complexity level for FreeRTOS kernel concepts
- On-board ST-LINK for flash and debug
- User LEDs and user button for scheduling and ISR labs
- Virtual COM Port for UART logs and timing observations
- STM32CubeIDE and STM32CubeMX project generation path

Boundaries for this course

- Use LED, button, UART, ST-LINK, SysTick, and FreeRTOS first.
- Keep Ethernet, USB, LCD-TFT, DMA2D, and crypto as later extension topics.
- Treat NUCLEO-F429ZI as a close board-family reference, not the main template.
- Keep NUCLEO-H755ZI-Q for later TinyML or Edge AI work.

Project Boundary Check

This checkpoint is deliberately light. The goal is conceptual readiness before writing FreeRTOS code.

Board and environment

1. Open STM32CubeIDE and confirm the NUCLEO-F439ZI board or STM32F439ZI MCU path.
2. List five board features visible in the ST board description.
3. Mark which features are needed for core FreeRTOS labs.
4. Mark which features should stay as future extension topics.

RTOS decision sketch

1. Invent a small firmware scenario with at least four activities.
2. Classify each activity as periodic, event-driven, continuous, or background.
3. Identify one timing risk in a super loop version.
4. Choose which activity would deserve the highest task priority and explain why.

Checkpoint output: a one-page note with workload categories, one timing risk, and a first task-priority hypothesis.

Bridge to Module 01

Module 00 defines the language. Module 01 opens the firmware and shows where the language appears in files.

From concept

- Task
- Scheduler
- Kernel object
- Configuration

To source boundary

- `main.c`
- `FreeRTOSConfig.h`
- Kernel source files
- Portable layer

To STM32 lab

- CubeMX-generated project
- HAL and BSP boundary
- ST-LINK debug
- UART observation path

After Module 00, Module 01 should spend less time justifying RTOS use and more time reading the concrete project structure.

References and Further Reading

References Used

- FreeRTOS, RTOS Fundamentals, Beginner's Guide.
- FreeRTOS, About FreeRTOS and Why FreeRTOS.
- FreeRTOS homepage, current project positioning and ecosystem notes.
- AWS FreeRTOS User Guide, What is FreeRTOS?
- AWS FreeRTOS User Guide, FreeRTOS kernel fundamentals.
- FreeRTOS Kernel Book, Chapter 1, Preface 1.1.1-1.1.6.

Further Reading

- FreeRTOS Kernel Book, Chapter 2, Source Organization.
- FreeRTOS Kernel Book, Chapter 4, Task Management.
- FreeRTOS API Reference, Task Control APIs.
- NUCLEO-F439ZI board documentation and STM32CubeIDE documentation.
- Course notes: STM32F439ZI lab environment and previous uC/OS-III material conversion memo.